

液态神经网络 (LNN) 深度解析：从神经科学原理到基础模型

液态神经网络(LNN)可视化网页

1 摘要

本报告将全面深入地探讨液态神经网络 (LNN) 这一新兴技术。我们将从其深刻的神经科学渊源——微小线虫 *C. elegans* 的神经系统——出发，详细阐述其核心的数学原理，即基于常微分方程 (ODE) 的连续时间动态。报告将追溯 LNN 架构的演进历程，从最初的液态时间常数网络 (LTC)，到计算高效的闭式连续时间模型 (CfC)，再到融合结构化状态空间模型的 Liquid-S4，直至最新的商业化液态基础模型 (LFM)。我们将 LNN 与循环神经网络 (RNN)、长短期记忆网络 (LSTM) 及 Transformer 等主流架构进行细致的比较分析，突出其在效率、鲁棒性和因果关系建模方面的独特优势与固有限制。最后，本报告将评估该技术当前的发展阶段，展望其在边缘计算、机器人和实时系统等领域的广阔前景与面临的挑战，并为希望实践该技术的研究者和开发者提供一份精选的开源项目指南。

2 液态神经网络的核心原理与灵感来源

2.1 神经科学的启示：从秀丽隐杆线虫到计算模型

液态神经网络 (Liquid Neural Networks, LNNs) 的设计哲学并非源于纯粹的数学抽象，而是深深植根于对生物智能的模拟，特别是对微小生物——秀丽隐杆线虫 (*Caenorhabditis elegans*) 神经系统的深入研究。理解这一生物学渊源，是掌握 LNN 核心思想及其独特优势的基石。

秀丽隐杆线虫是一种仅长约 1 毫米的模式生物，其整个神经系统仅由 302 个神经元和大约 8000 个突触连接构成。尽管结构如此简单，它却能展现出移动、觅食、学习和对环境做出反应等一系列复杂的自适应行为。这种“以小博大”的惊人效率，成为了 LNN 研究的最初灵感来源。研究人员观察到，这种微小生物的运动能力甚至超越了当时任何先进的机器人系统。这一现象引发了一个核心问题：能否通过模仿生物神经系统的设计原则，来构建更高效、更鲁棒的人工智能模型？

对 *C. elegans* 的研究揭示了几个关键的启发点，这些启发点直接塑造了 LNN 的架构：

- 极致的参数效率：***C. elegans* 证明了用极少的神经元实现复杂功能是完全可能的。这直接启发了 LNN 追求参数效率的设计哲学，旨在用远比传统深度学习模型更小、更紧凑的网络来解决复杂的时序任务。
- 连续时间动态处理：**生物神经系统中的信号传递是一个连续的、模拟的过程，而非计算机中离散的时钟节拍。LNN 为了模拟这种生物真实性，采用了常微分方程 (Ordinary Differential Equations, ODEs) 来描述其神经元状态的连续演化，这与传统人工神经网络 (ANN) 基于离散时间步长 (discrete time-steps) 进行更新的范式形成了根本性区别。
- 动态与适应性：**线虫神经元的反应不是一成不变的。对于相同的输入信号，其反应会受到历史状态和当前上下文的影响，表现出非线性、概率性和时间依赖性的特征。LNN 通过其核心的“液态”特

性——即动态变化的内部参数——来模拟这种灵活的突触反应机制，使其能够根据输入数据的动态变化而调整自身的行为。

这种对生物拟真性的追求，构成了 LNN 区别于当前主流 AI 范式的核心差异化优势。在“规模就是一切”（Scale is all you need）的思潮下，LNN 提出了一条不同的路径：通过更智能、更接近生物机理的“巧设计”，可以在远小于巨型模型的规模上实现高水平的智能。研究人员观察到，

C. elegans 在充满噪声和变化的真实环境中表现出极强的鲁棒性和适应性。他们假设，这种卓越能力源于其神经系统固有的连续、动态的工作方式。因此，通过在计算模型中复现这些特性，LNN 有望在鲁棒性、分布外（out-of-distribution）泛化以及因果推理方面，内生地优于那些主要依赖海量数据进行统计模式拟合的传统模型。

2.2 数学基础：连续时间动态与常微分方程

从数学上看，LNN 被严谨地定义为一类由常微分方程（ODE）描述的连续时间循环神经网络（Continuous-Time Recurrent Neural Network, CT-RNN）。这与我们熟知的传统 RNN 架构在根本上有所不同。

传统的 RNN，包括其变体长短期记忆网络（LSTM）和门控循环单元（GRU），其隐藏状态是在离散的时间步长 $t, t+1, t+2, \dots$ 上进行更新的。其核心是一个递归关系式，形式为 $h_t = f(h_{t-1}, x_t)$ ，其中 h_t 是在时间步 t 的隐藏状态， h_{t-1} 是前一时刻的状态， x_t 是当前时刻的输入。

相比之下，LNN 的隐藏状态 $x(t)$ 是时间变量 t 的一个连续函数。它的变化不是通过离散的迭代更新，而是由一个微分方程来描述其随时间的变化率（即导数）。这个方程的一般形式为：

$$\frac{dx(t)}{dt} = f(x(t), I(t), t, \theta)$$

其中， $x(t)$ 是隐藏状态向量， $I(t)$ 是输入信号， t 是时间， θ 是网络的可学习参数。由于这个微分方程通常是非线性的，一般不存在直接的解析解。因此，要计算在任意时间点

T 的隐藏状态 $x(T)$ ，需要从一个初始状态 $x(0)$ 开始，使用数值 ODE 求解器（如欧拉法、龙格-库塔法等）对方程进行积分模拟。

这种连续时间的框架赋予了 LNN 一个天然的优势，即能够优雅地处理不规则采样的时间序列数据。在现实世界中，许多数据流，如医疗监控中的心电图信号、工业设备的传感器读数或金融交易记录，其数据点在时间上并非均匀分布。传统 RNN 的离散步长假设在这种情况下显得力不从心，通常需要对数据进行插值或填充等预处理，这不仅过程繁琐，还可能引入人为的噪声和偏差，甚至丢失关键的瞬时动态信息。

LNN 的 ODE 公式则可以自然地在任意时间点

t 上进行求解，无缝地处理不规则的时间间隔，因为时间 t 本身就是模型的一个内在变量。这使得 LNN 在处理真实物理世界和生物信号方面具有根本性的架构优势，成为机器人技术、自动驾驶和医疗健康等领域进行时间序列分析的理想候选。

2.3 “液态”的精髓：可变的神经元时间常数

LNN 中“液态”（Liquid）一词的由来，并非一个模糊的比喻，而是具有精确的数学含义。它特指模型中神经元的时间常数（time-constant, τ ）是动态可变的，而非一个固定的超参数。这是 LNN 区别于其他 CT-RNN 的核心创新之处。

在作为 LNN 奠基之作的液态时间常数网络（Liquid Time-Constant Networks, LTC）中，其隐藏状态由一个精心设计的 ODE 系统描述。其具体形式如下：

$$\frac{dx(t)}{dt} = - \left(\frac{1}{\tau_{base}} + f(x(t), I(t), \theta) \right) x(t) + f(x(t), I(t), \theta) A$$

在这个方程中， $f(x(t), I(t), \theta)$ 是一个由当前隐藏状态 $x(t)$ 和输入 $I(t)$ 控制的小型神经网络，通常实现为一个门控机制。 τ_{base} 和 A 是可学习的参数。

这里的关键在于，系统的有效时间常数 $\tau_{effective}$ 动态地依赖于神经网络 f 的输出。在物理学和动态系统中，时间常数 τ 决定了系统状态衰减到其平衡点的速度，从而控制了神经元的“记忆”跨度和对输入的响应速度。在 LTC 中，这个时间常数不再是固定的，而是“液化”的，它会根据输入信号的特性实时调整。

这种动态时间常数是 LNN 实现自适应计算的关键机制。在传统模型中，固定的时间常数意味着网络以一种固定的模式处理所有时间动态。而“液态”时间常数赋予了网络一种前所未有的灵活性：当输入信号变化剧烈时，网络可以通过其内部的门控机制 f 动态地减小其有效时间常数，从而变得更敏感、反应更快、记忆更短；反之，当输入信号平稳、需要捕捉长期依赖关系时，它可以增大时间常数，以整合更长时间维度的信息。这种能力极大地增强了模型的“表达能力” (expressivity)。实验表明，LNN 能够生成比具有固定动态的模型更复杂、更丰富的状态轨迹，从而能够拟合更复杂的数据模式，更好地理解数据背后的因果结构。

3 LNN 架构的演进之路

液态神经网络自诞生以来，经历了一条快速而清晰的演进路径，每一代架构的出现都旨在解决前代的核心局限，并拓展其应用边界。

3.1 LTC (液态时间常数网络): 奠基之作

液态时间常数网络 (LTC) 是 LNN 的第一个具体实现，由 Ramin Hasani 及其同事在 2020 年 AAAI 会议的论文中正式提出，并于 2021 年发表。LTC 是整个 LNN 技术族的理论基石，它完整地实现了基于 ODE 和动态时间常数的核心思想。

LTC 的核心在于其前向传播过程严格依赖数值 ODE 求解器来模拟神经元状态的连续演化。这一设计在数学上是严谨的，确保了模型能够忠实地反映其底层动态系统的行为。通过理论分析，研究者证明了 LTC 动态系统具有稳定和有界的特性，并且在表达能力上优于其他类型的神经 ODE 模型。在一系列时间序列预测基准测试中，LTC 也展示了相较于经典和现代 RNN 的优越性能。

然而，LTC 的成功也伴随着一个致命的局限：计算效率。对数值 ODE 求解器的严重依赖导致其训练和推理速度极其缓慢，计算成本高昂。求解器需要在每个时间步内部进行多次迭代计算，这带来了巨大的开销，极大地限制了 LTC 在需要实时响应和大规模部署的场景下的实际应用价值。

3.2 CfC (闭式连续时间模型): 效率的革命

为了克服 LTC 的计算瓶颈，研究团队提出了闭式连续时间模型 (Closed-form Continuous-time models, CfC)，其核心目标就是将 LNN 从一个理论上优雅但实践中缓慢的模型，转变为一个高效、实用的工具。

CfC 的核心创新在于，研究者们通过精妙的数学推导，为 LTC 动态方程中的一个关键积分项找到了一个高质量的近似闭式解 (analytical approximation)。这个突破意味着，原本需要通过数值求解器进行多步迭代计算的 ODE 过程，可以被一个紧凑、可直接计算的神经网络层所替代。本质上，CfC 是用一个高效的、非迭代的函数逼近器来模拟 ODE 求解器的行为，同时保留了 LNN 核心的动态特性。

这一创新带来了革命性的成果。在速度上，CfC 的训练和推理比基于 ODE 的 LTC 模型快了 1 到 5 个数量级 (即 10 倍到 10 万倍)。更重要的是，这种巨大的速度提升并非以牺牲性能为代价。在多个标准序

列基准测试中，CfC 的准确率与 LTC 相比损失通常不到 2%，几乎完整地保留了 LTC 强大的表达能力。

CfC 的出现可以被视为 LNN 发展史上的一个“工程奇迹”。它是一个典型的在理论与工程之间做出成功权衡的案例，通过牺牲数学上的绝对精确性（从精确的数值解到高效的近似解析解），换来了数量级的性能提升。这一飞跃使得 LNN 技术成功跨越了从学术论文到实际应用之间的鸿沟，使其能够被成功部署在资源受限的边缘设备上（如微控制器、移动 SoC），并进行实时处理，从而极大地扩展了其潜在的应用范围和商业价值。

3.3 Liquid-S4 (液态结构化状态空间模型): 拥抱长程依赖

随着模型的发展，研究者们开始寻求将 LNN 的动态适应性与当时在处理长程依赖方面表现最出色的架构——结构化状态空间模型 (Structured State-Space Model, S4)——进行结合。其目标是创建一个既能动态适应输入，又能高效捕捉极长序列依赖关系的新模型，这便是 Liquid-S4 的由来。

Liquid-S4 的创新之处在于，它并非简单地将两种架构进行堆叠，而是将一个线性化版本的 LTC 动态方程 $\dot{x} = (A + Bu)x + Bu$ 巧妙地嵌入到 S4 模型的数学框架之内。这种深度融合产生了一个额外的、输入依赖的卷积核，研究者称之为“液态核” (liquid kernel)。这个特殊的核能够捕捉输入信号中不同时间点之间的自相关性，从而进一步增强了模型的表达能力。

实验结果证明了这种融合策略的巨大成功。在极具挑战性的长程竞技场 (Long Range Arena, LRA) 基准测试中，Liquid-S4 在文本、语音和图像序列等多个任务上取得了超越原始 S4 和众多 Transformer 变体的 SOTA 性能。

Liquid-S4 的诞生标志着 LNN 技术发展战略的一次重要转变——从一个寻求替代传统模型的“独立方案”，演变为一个可以协同增强其他强大架构的“功能模块”。这表明 LNN 的研究团队没有固步自封，而是积极地将 LNN 的核心思想（如输入依赖的动态系统）与当时最前沿的技术进行融合。这种策略巧妙地弥补了纯 LNN 作为 RNN 变体在处理极长序列时可能面临的梯度和记忆瓶颈，同时将 S4 强大的长程记忆力与 LNN 的动态适应性相结合，最终实现了 1+1>2 的协同效应。

3.4 LFM (液态基础模型): 走向商业化与大规模

LNN 演进的最新阶段是由其核心研究人员（包括 Ramin Hasani）创立的 Liquid AI 公司所主导的商业化努力，其成果便是液态基础模型 (Liquid Foundation Models, LFM)。LFM 的目标是将 LNN 的核心原理进行规模化，打造出能够与 GPT 和 Claude 等主流大模型竞争的新一代基础模型。

LFM 的核心创新体现在以下几个方面：

- 1. 规模化：**Liquid AI 推出了从 13 亿 (LFM-1.3B) 到 400 亿 (LFM-40B) 参数不等的一系列模型，旨在全面覆盖从资源受限的边缘设备到强大的云端数据中心的部署需求。
- 2. 架构融合：**LFM 在 LNN 的核心原理之上，集成了现代大语言模型的关键技术，特别是 **专家混合 (Mixture of Experts, MoE)** 架构。这使得模型能够在急剧扩大参数规模的同时，通过稀疏激活的方式保持计算效率。
- 3. 长上下文优化：**LFM 的核心市场卖点是解决了 Transformer 架构在处理长序列时面临的 KV 缓存 (KV cache) 瓶颈问题。LFM 声称，在处理长输入序列时，其推理时间和内存占用几乎是恒定的，能够以极高的效率支持长达 32k tokens 的上下文窗口，这对于传统 Transformer 而言是极具挑战性的。

LFM 的出现体现了 LNN 核心思想在商业逻辑驱动下的重新包装和战略演进。它一方面继承并放大了 LNN “高效” 的基因，并以此作为对抗 Transformer 架构在成本和部署灵活性方面的核心优势；另一方

面，它又通过拥抱“规模”和 MoE 等技术，积极参与到大模型性能的激烈竞争中。这一战略精准地瞄准了市场的核心痛点：即业界迫切需要既能处理长上下文又能在各种硬件（尤其是边缘设备）上高效运行的大模型。LFM 的闭源商业模式也标志着该技术从早期的开放研究阶段，正式迈向了成熟的专有产品阶段。

4 与传统神经网络的比较分析

为了更清晰地定位液态神经网络，本节将其与两种主流的序列处理架构——循环神经网络（RNN/LSTM）和 Transformer——进行详细的比较。

4.1 与循环架构的对比 (RNN, LSTM, GRU)

LNN 本身可以看作是 RNN 家族的一个高级成员，但它在几个关键方面与传统的 RNN、LSTM 和 GRU 有着本质的不同。

- **时间处理方式**：LNN 是连续时间模型，其底层的 ODE 公式使其能够原生处理带有不规则和非均匀时间戳的时间序列数据。而 RNN/LSTM/GRU 是离散时间模型，它们假设输入序列的时间步是等间隔的。当面对不规则数据时，这些模型通常需要进行插值或填充等预处理，这可能会引入人为的偏差。
- **梯度问题与表达能力**：传统 RNN 面临严重的梯度消失/爆炸问题，难以学习长期依赖。LSTM 和 GRU 通过其精巧的门控机制（如遗忘门、输入门等）极大地缓解了这一问题，成为处理序列任务的标准。LNN 通过其动态系统特性，在一定程度上也缓解了梯度问题，但作为 RNN 的变体，并非完全免疫。然而，LNN 的动态时间常数赋予了其比结构固定的 LSTM/GRU 更强的理论表达能力，使其能够模拟更复杂的动态系统。
- **因果性建模**：由于 LNN 的设计源于对生物和物理动态系统的模拟，它被认为能更好地捕捉数据背后潜在的因果关系，而不仅仅是统计上的相关性。这使得 LNN 在需要理解“为何”做出决策的任务中更具优势。

4.2 与 Transformer 架构的对比

与 Transformer 相比，LNN 代表了另一条截然不同的技术路线。

- **核心计算范式**：LNN 本质上是序贯处理（Recurrent）模型，其当前状态依赖于前一时刻的状态。而 Transformer 是基于自注意力机制（Self-Attention）的并行处理模型，它可以一次性地计算序列中所有元素之间的相互关系，无需按顺序进行。
- **计算与内存复杂度**：这是两者最显著的区别。
 - **Transformer**：其自注意力机制的计算复杂度与序列长度 L 的平方成正比 ($O(L^2 \cdot D)$)，其中 D 是隐藏层维度。在推理（生成）阶段，它需要一个 KV 缓存来存储历史信息，该缓存的大小与序列长度 L 线性相关 ($O(L \cdot D)$)。这使得 Transformer 在处理非常长的序列时，计算和内存成本会急剧上升，成为其应用的主要瓶颈。
 - **LNN (特别是 Cfc 及后续变体)**：其循环结构的计算复杂度与序列长度 L 呈线性关系 ($O(L \cdot D)$)。更关键的是，在推理阶段，它的时间和内存复杂度几乎与序列长度无关（近乎 $O(D)$ ）。这是因为它不需要像 Transformer 那样存储整个历史序列的键值对，只需维持一个固定大小的隐藏状态即可。这一特性使其在长上下文处理和边缘设备部署上具有巨大的效率优势。
- **并行性与训练效率**：Transformer 的高度并行性使其能够充分利用现代 GPU 的强大算力，从而在大型数据集上实现高效的训练。相比之下，LNN 的序贯处理特性限制了其并行能力，这可能导致其训练时间相对较长。不过，像 Liquid-S4 这样的混合架构通过将 LNN 的动态转化为卷积形式，在一定程度上解决了这个问题。

- **适用场景：**基于上述差异，两种架构形成了互补的优势领域。Transformer 主导了需要大规模并行训练和强大长程依赖建模的云端自然语言处理任务。而 LNN 则在资源受限的边缘 AI、实时控制系统（如机器人、自动驾驶）以及需要处理不规则、连续时序数据的场景中展现出独特的价值。

4.3 架构比较总结表

下表直观地总结了 LNN、RNN/LSTM 和 Transformer 在关键特性上的核心差异，为技术选型提供了清晰的参考。

特性 (Feature)	液 态 神 经 网 络 (LNN - CfC/LTC)	循 环 神 经 网 络 (RNN/LSTM)	Transformer
核 心 机 制	连续时间动态系统 (ODE)	离散时间递归状态更新	并行自注意力机制
时 间 数 据 处 理	原生支持连续和不规则时间	假设离散、等间隔时间步	通过位置编码注入时序信息
长 程 依 赖	通过动态时间常数调节，但可能存在梯度问题。Liquid-S4 显著增强此能力。	LSTM/GRU 缓解梯度消失，但对极长序列仍具挑战性。	通过自注意力机制直接访问所有位置，能力强。
并 行 性	序贯处理，并行性差 (Liquid-S4 除外)	序贯处理，并行性差	高度并行，训练效率高
计 算 复 杂 度 (推 理)	$O(L \cdot D^2)$ (LTC) 或 $O(L \cdot D)$ (CfC)	$O(L \cdot D^2)$	$O(L^2 \cdot D)$
内 存 复 杂 度 (推 理)	近乎 $O(D)$ (无 KV 缓存)	$O(D)$	$O(L \cdot D)$ (KV 缓存)
参 数 效 率	极高，用少量神经元完成复杂任务	中等	参数量巨大
内 在 因 果 性	强，源于动态系统本质	弱，序贯性提供时序因果	弱，需从数据中学习关系

导出到 Google 表格

5 发展现状、前景与挑战

5.1 当前发展阶段

液态神经网络技术的发展路径异常迅速且清晰。它从 2020 年的一个学术概念 (LTC) 出发，在短短几年内迅速演进：2021 年，通过 CfC 实现了高效的工程实现；2022 年，通过 Liquid-S4 与 SOTA 架构深度融合，在性能上达到新高度；最终在 2024 年左右，以 LFM 的形式实现了全面的商业化。这一历程表明，LNN 的核心技术已经历了多轮迭代和验证，达到了可以产品化的成熟度。

然而，尽管核心技术发展迅速，其周边的开发者生态系统——包括标准化的库、丰富的教程、广泛的社区支持以及与第三方工具的集成——仍然远不及 TensorFlow 和 PyTorch 生态系统中的 Transformer。目前，除了少数由核心研究团队维护的开源库外，缺乏像 Hugging Face 之于 Transformer 那样成熟、易用的生态平台，这成为其进一步推广和普及的主要障碍之一。

5.2 技术优势与未来前景 (The Good)

LNN 的独特属性为其在多个前沿领域开辟了广阔的应用前景。

- 边缘 AI 革命：**LNN 的极致参数效率和低计算/内存消耗，使其成为边缘计算的理想选择。在自动驾驶、机器人控制、可穿戴健康设备和工业物联网 (IIoT) 等领域，LNN 有潜力在本地硬件上进行实时、低功耗的智能决策，无需持续依赖云端连接，从而降低延迟、保护数据隐私并提高系统可靠性。
- 构建鲁棒与安全的 AI：**在充满噪声和扰动的现实世界应用中，LNN 已被证明比传统模型表现出更强的鲁棒性。例如，在自动驾驶的模拟测试中，即使在视觉环境发生剧烈变化（如季节更替）或存在噪声干扰时，LNN 的注意力也能更稳定地集中在关键的因果特征（如道路边界或目标车辆）上，而传统模型的注意力图则容易受到无关背景（如路边树木）的干扰。这种特性对于安全关键系统（如医疗诊断、自动驾驶）至关重要。
- 探索因果与可解释 AI 的新路径：**LNN 的动态系统本质使其有望构建更具因果推理能力和可解释性的模型。与被视为“黑箱”的许多深度模型不同，通过分析 LNN 内部状态的动态演化轨迹，工程师和研究人员可能更容易理解模型“为何”做出某个决策，这有助于诊断模型故障、增强系统信任度。
- 开创长上下文处理的新范式：**以 LFM 为代表的最新 LNN 架构，通过解决 Transformer 的 KV 缓存瓶颈，为处理超长文档、视频流、基因序列等任务提供了比现有方案更具成本效益的解决方案。这有望在科学计算、金融分析、法律文书处理和长视频内容创作等领域开辟全新的应用可能性。

5.3 局限与未来挑战 (The Bad and The Ugly)

尽管前景光明，LNN 的推广和应用仍面临一系列挑战。

- 训练复杂性与超参数调整：**虽然 CfC 极大地提升了推理速度，但 LNN 的训练过程仍然可能比传统模型更具挑战性。其动态系统的性质要求对参数进行仔细的初始化和调整，其超参数搜索空间可能更为复杂，对使用者的经验和调试技巧要求更高。
- 长程依赖的根本性挑战：**作为一个继承了 RNN 血统的架构，纯粹的 LNN 在处理极长序列依赖时，仍然可能面临梯度不稳定（消失或爆炸）的根本性问题，尽管其内部机制对此有所缓解。Liquid-S4 和 LFM 的混合架构设计，本身就说明了研究者们正在积极地通过融合其他技术（如 SSM 或 MoE）来弥补这一潜在的短板。
- 生态系统与标准化缺失：**如前所述，缺乏成熟的生态系统是 LNN 大规模普及的最大障碍之一。开发者迫切需要更多标准化的工具、丰富的预训练模型库 (Model Zoo) 和便捷的部署方案，以降低使用门槛。
- 可解释性的承诺与现实：**虽然 LNN 在理论上比传统黑箱模型更具可解释性，但在一个包含成千上万个相互作用的非线性动态单元的复杂 LNN 模型中，要完全理解其内部状态的演化仍然是一个巨大的挑战，对于非领域专家的技术门槛非常高。

5.4 性能基准测试结果

下表汇总了 LNN 及其变体在多个关键基准测试任务上的量化性能表现，为上述的优势论述提供了具体的证据支持。

任务 (Task)	模型 (Model)	关键结果 (Key Result)	参数/效率 (Parameters/Efficiency)	来源 (Source)
自动驾驶 (车道保持)	LNN	性能与需要超过 10 万个神经元的传统模型相当	仅用 19 个神经元	
无人机目标跟踪	LNN	在不同季节环境下仍能准确跟踪目标，展现出	34 个神经元, 12,000 个连接	

任务 (Task)	模型 (Model)	关键结果 (Key Result)	参数/效率 (Parameters/Efficiency)	来源 (Source)
		强大的分布外泛化能力		
人类活动识别	CfC	准确率优于基线模型几个百分点	速度比最佳 ODE 模型快 8,752%	
物理动态建模 (Walker2D)	CfC	性能击败包括 Transformer 在内的基线模型	速度显著提升	
IMDB 情感分析	CfC (混合记忆)	性能优于其他 RNN 模型	-	
PhysioNet 2012 (医疗)	CfC	性能具有竞争力	比其他连续模型快 160-220 倍	
Speech Commands (语音)	Liquid-S4	准确率 96.78% (SOTA)	参数比 S4 少 30%	
Long Range Arena (LRA)	Liquid-S4	平均准确率 87.32% (SOTA)	-	

6 开源项目与学习资源

对于希望学习和实践液态神经网络的开发者和研究人员，社区已经提供了一些宝贵的开源资源。

6.1 官方与核心代码库

这些是由 LNN 核心研究团队发布和维护的代码库，是理解和复现其工作的最权威来源。

- **LTC 原始实现**: github.com/raminmh/liquid_time_constant_networks 。这是理解 LNN 起源的必看项目，基于较早的 TensorFlow 1. x。对于希望追本溯源的研究者来说，这是最重要的起点。
- **CfC 官方实现**: github.com/raminmh/CfC 。该库提供了 TensorFlow 2. x 和 PyTorch 的实现，是学习和使用高效 CfC 模型的关键资源，也是目前应用最广泛的版本之一。
- **Liquid-S4 官方实现**: github.com/raminmh/liquid-s4 。这个代码库基于 PyTorch Lightning 和 Hydra 等高级框架构建，结构较为复杂，主要面向希望复现 SOTA 结果或进行前沿研究的进阶用户。
- **LFM 开源尝试**: github.com/kyegomez/LFM 。需要注意的是，Liquid AI 的官方 LFM 是闭源的。此项目是社区对 LFM 架构的开源尝试和复现，对于理解其可能的设计思想具有参考价值。

这些代码库的演进也清晰地反映了整个机器学习社区开发实践的变迁：从 LTC 的 TensorFlow 1，到 CfC 的同时支持 TensorFlow 2/PyTorch，再到 Liquid-S4 全面转向基于 PyTorch Lightning 和 Hydra 的高级框架。这为学习者提供了一个有趣的视角来观察技术生态的发展。

6.2 社区应用项目示例

除了官方库，社区也涌现出一些基于 LNN 的应用项目，它们是绝佳的学习案例。

- **入门级（分类、聚类与回归）**: github.com/SeyedMuhammadHosseinMousavi/Liquid-Neural-Networks-LNNs-Classification 。这是一个使用 LTC 在经典的 Iris 数据集上进行多种任务的入门级项目。其代码清晰易懂，非常适合初学者上手，以理解 LNN 的基本用法。

- **实际应用（股票市场预测）**：github.com/HusseinJammal/Liquid-Neural-Networks-in-Stock-Market-Prediction。这是一个更具实际意义的应用案例，展示了如何将 LNN 用于金融时间序列预测，项目中包含了数据获取、特征工程和模型评估等完整流程。
- **跨领域应用（图像分类）**：github.com/babycommando/liquid-neural-networks。该项目展示了如何将本质上是序列模型的 LNN 用于图像分类（通过将图像展平为像素序列），这是一个理解模型灵活性和适用边界的绝佳例子。
- **企业级部署（云平台集成）**：github.com/fg-research/lnn-sagemaker。这个项目展示了如何在 AWS SageMaker 云平台上打包和部署 LNN 进行时间序列预测，对于有实际生产部署需求的工程师来说，这是一个非常有价值的参考。

6.3 学习路径建议

为系统地学习 LNN 技术，建议遵循以下路径：

1. **理论入门**：首先应阅读 LNN 的奠基性论文，特别是 LTC 的论文（arXiv: 2006.04439），以理解其核心思想；然后阅读 CfC 的论文（arXiv: 2106.13898），以了解其如何实现效率突破。
2. **基础实践**：从简单的社区项目入手，例如在本地环境中运行 Iris 分类或 MNIST 分类的代码，亲手调试并理解数据流和模型结构。
3. **深入研究**：克隆并尝试复现官方 CfC 代码库中的实验，调整超参数，深入理解其 PyTorch 或 TensorFlow 的实现细节。
4. **前沿探索**：对于高级用户和研究者，可以挑战研究 Liquid-S4 的代码库，并尝试将其强大的长序列建模能力应用于自己的研究课题或复杂任务中。
5. **关注商业动态**：持续关注 Liquid AI 公司的官方博客和产品发布，了解 LNN 技术商业化的最新进展和应用案例，这有助于把握技术发展的未来方向。

7 结论

液态神经网络代表了人工智能领域中一条独特的、受生物启发的演进路径。它通过模拟生物神经系统的连续时间动态和自适应神经元特性，在计算效率、环境鲁棒性和对现实世界动态的建模能力方面，为传统的、基于离散时间和静态结构的深度学习范式提供了有力的补充，甚至在某些领域构成了直接的挑战。

从 LTC 的理论雏形，到 CfC 的效率革命，再到 Liquid-S4 的性能巅峰和 LFM 的商业化落地，LNN 在短短数年内展现了惊人的发展速度和潜力。这一演进路径清晰地表明，该技术正从一个学术界的有趣思想，快速转变为能够解决实际工业问题的成熟工具。

展望未来，LNN 最光明的应用前景在于赋能下一代边缘 AI 和实时智能系统，这些领域对模型的效率、功耗和鲁棒性有着极为苛刻的要求。然而，它若想成为与 Transformer 并驾齐驱的主流架构，还必须克服**生态系统不成熟、训练门槛相对较高等挑战**。液态神经网络的未来，将不仅取决于其自身核心技术的持续迭代，更依赖于整个开发者社区的接纳、共建和创新应用。

8 参考

1. Liquid Neurons and Neural Worms: A Cognitive Neuroscience Approach for Advanced Deep Learning and AI - Community.aws, 访问时间为 六月 22, 2025, <https://community.aws/content/2pFGxH4YdQNerVQT2d8IBBOhy6x/liquid-neurons-and-neural-worms-a-cognitive-neuroscience-approach-for-advanced-deep-learning-and-ai>
2. The brain of a tiny worm inspired a new type of AI - Science News Explores, 访问时间为 六月 22, 2025, <https://www.snexplores.org/article/the-brain-of-a-tiny-worm-inspired-a-new-type-of-ai-liquid-neural-network>
3. Researchers Discover a More Flexible Approach to Machine ..., 访问时间为 六月 22, 2025, <https://www.quantamagazine.org/researchers-discover-a-more-flexible-approach-to-machine-learning-20230207/>
4. Liquid Neural Networks: Edge Efficient AI (2025) - Ajith's AI Pulse, 访问时间为 六月 22, 2025, <https://ajithp.com/2025/05/04/liquid-neural-networks-edge-ai/>
5. The Future of Machine Learning is Liquid - devmio, 访问时间为 六月 22, 2025, <https://devm.io/machine-learning/liquid-machine-learning>
6. Summary of Liquid Neural Networks - Summarize.ing, 访问时间为 六月 22, 2025, <https://summarize.ing/video-25369-Liquid-Neural-Networks>
7. Liquid Neural Networks | Ramin Hasani | TEDxMIT - YouTube, 访问时间为 六月 22, 2025, <https://www.youtube.com/watch?v=RI35E5ewBul&pp=0gcJCdgAo7VqN5tD>
8. Liquid Neural Networks | The Center for Brains, Minds & Machines - CBMM @ MIT, 访问时间为 六月 22, 2025, <https://cbmm.mit.edu/video/liquid-neural-networks>
9. Liquid Time-constant Networks, 访问时间为 六月 22, 2025, <https://ojs.aaai.org/index.php/AAAI/article/view/16936/16743>
10. Liquid Time-constant Networks - Association for the Advancement of Artificial Intelligence (AAAI), 访问时间为 六月 22, 2025, <https://cdn.aaai.org/ojs/16936/16936-13-20430-1-2-20210518.pdf>
11. RNN vs LSTM vs GRU vs Transformers - GeeksforGeeks, 访问时间为 六月 22, 2025, <https://www.geeksforgeeks.org/rnn-vs-lstm-vs-gru-vs-transformers/>
12. Recurrent neural network - Wikipedia, 访问时间为 六月 22, 2025, https://en.wikipedia.org/wiki/Recurrent_neural_network
13. (PDF) Liquid Time-constant Networks - ResearchGate, 访问时间为 六月 22, 2025, https://www.researchgate.net/publication/342027480_Liquid_Time-constant_Networks
14. Liquid Neural Networks: Fluid, Flexible Neurons - Deepgram, 访问时间为 六月 22, 2025, <https://deepgram.com/learn/liquid-neural-networks>

15. arXiv:2006.04439v4 [cs.LG] 14 Dec 2020, 访问时间为 六月 22, 2025,
<http://arxiv.org/pdf/2006.04439>
16. raminmh/liquid_time_constant_networks: Code Repository ... - GitHub, 访问时间为 六月 22, 2025, https://github.com/raminmh/liquid_time_constant_networks
17. Liquid Time-constant Networks | Request PDF - ResearchGate, 访问时间为 六月 22, 2025,
https://www.researchgate.net/publication/363386857_Liquid_Time-constant_Networks
18. [2006.04439] Liquid Time-constant Networks - arXiv, 访问时间为 六月 22, 2025,
<https://arxiv.org/abs/2006.04439>
19. Liquid Time-constant Networks | Proceedings of the AAAI Conference on Artificial Intelligence, 访问时间为 六月 22, 2025,
<https://ojs.aaai.org/index.php/AAAI/article/view/16936>
20. Closed-form Continuous-time Neural Networks arXiv:2106.13898v2 [cs.LG] 2 Mar 2022, 访问时间为 六月 22, 2025, <http://arxiv.org/pdf/2106.13898>
21. Closed-form Continuous-time Neural Networks arXiv:2106.13898v2 ..., 访问时间为 六月 22, 2025, <https://arxiv.org/abs/2106.13898>
22. Closed-form continuous-time neural networks - ISTA Research Explorer, 访问时间为 六月 22, 2025, <https://research-explorer.ista.ac.at/record/12147>
23. FedCFC: On-Device Personalized Federated Learning with Closed-Form Continuous-Time Neural Networks - GitHub Pages, 访问时间为 六月 22, 2025,
<https://tanrui.github.io/pub/FedCFC.pdf>
24. LIQUID STRUCTURAL STATE-SPACE MODELS - OpenReview, 访问时间为 六月 22, 2025,
<https://openreview.net/pdf?id=g4OTKRKfS7R>
25. Liquid Structural State-Space Models - arXiv, 访问时间为 六月 22, 2025,
<https://arxiv.org/pdf/2209.12951>
26. Liquid Structural State-Space Models, 访问时间为 六月 22, 2025,
<https://arxiv.org/abs/2209.12951>
27. Liquid Foundation Models: Our First Series of Generative AI Models ..., 访问时间为 六月 22, 2025, <https://www.liquid.ai/blog/liquid-foundation-models-our-first-series-of-generative-ai-models>
28. Liquid AI: Build efficient general-purpose AI at every scale., 访问时间为 六月 22, 2025,
<https://www.liquid.ai/>
29. Liquid Foundation Models | Liquid AI, 访问时间为 六月 22, 2025,
<https://www.liquid.ai/models>

30. Liquid AI Introduces Liquid Foundation Models (LFMs): A 1B, 3B, and 40B Series of Generative AI Models - MarkTechPost, 访问时间为 六月 22, 2025, <https://www.marktechpost.com/2024/10/03/liquid-ai-introduces-liquid-foundation-models-lfms-a-1b-3b-and-40b-series-of-generative-ai-models/>
31. Decentralised-AI/LFM-Liquid-AI-Liquid-Foundation-Models: An open source implementation of LFMs from Liquid AI - GitHub, 访问时间为 六月 22, 2025, <https://github.com/Decentralised-AI/LFM-Liquid-AI-Liquid-Foundation-Models>
32. Our Solutions | Liquid AI, 访问时间为 六月 22, 2025, <https://www.liquid.ai/solutions>
33. kyegomez/LFM: An open source implementation of LFMs from Liquid AI: Liquid Foundation Models - GitHub, 访问时间为 六月 22, 2025, <https://github.com/kyegomez/LFM>
34. Types of Deep Neural Networks - MRI Questions, 访问时间为 六月 22, 2025, <https://mriquestions.com/deep-network-types.html>
35. RNNs vs LSTM vs Transformers | SabrePC Blog, 访问时间为 六月 22, 2025, <https://www.sabrepc.com/blog/deep-learning-and-ai/rnns-vs-lstm-vs-transformers>
36. Liquid neural networks: A neuro-inspired revolution in AI and Robotics - RoboticsBiz, 访问时间为 六月 22, 2025, <https://roboticsbiz.com/liquid-neural-networks-a-neuro-inspired-revolution-in-ai-and-robotics/>
37. Why does the transformer do better than RNN and LSTM in long-range context dependencies? - AI Stack Exchange, 访问时间为 六月 22, 2025, <https://ai.stackexchange.com/questions/20075/why-does-the-transformer-do-better-than-rnn-and-lstm-in-long-range-context-depen>
38. Distributed Robotics Laboratory - MIT CSAIL, 访问时间为 六月 22, 2025, <https://www.csail.mit.edu/research/distributed-robotics-laboratory>
39. Disadvantages of Neural Networks/Deep Learning - why are they not preferred in all situations? : r/MachineLearning - Reddit, 访问时间为 六月 22, 2025, https://www.reddit.com/r/MachineLearning/comments/3dnolr/disadvantages_of_neural_networksdeep_learning_why/
40. raminh/CfC: Closed-form Continuous-time Neural Networks - GitHub, 访问时间为 六月 22, 2025, <https://github.com/raminh/CfC>
41. raminh/liquid-s4: Liquid Structural State-Space Models - GitHub, 访问时间为 六月 22, 2025, <https://github.com/raminh/liquid-s4>
42. GitHub - SeyedMuhammadHosseinMousavi/Liquid-Neural-Networks-LNNs-Classification, 访问时间为 六月 22, 2025, <https://github.com/SeyedMuhammadHosseinMousavi/Liquid-Neural-Networks-LNNs-Classification>
43. HusseinJammal/Liquid-Neural-Networks-in-Stock-Market-Prediction - GitHub, 访问时间为 六月 22, 2025, <https://github.com/HusseinJammal/Liquid-Neural-Networks-in-Stock-Market->

Prediction

44. liquid-neural-networks/liquidMNIST.ipynb at main - GitHub, 访问时间为六月 22, 2025, <https://github.com/babycommando/liquid-neural-networks/blob/main/liquidMNIST.ipynb>
45. SageMaker implementation of liquid neural networks (LNNs) for time series forecasting. - GitHub, 访问时间为六月 22, 2025, <https://github.com/fg-research/lnn-sagemaker>**